

Backward compatibility

Allows for data produced with an older schema to be read as though it were produced with a newer schema. This enables a consumer of data to use a newer schema to read older data. There are several scenarios where this is particularly useful:

- The consumer is expecting a new feature to be delivered by the upstream team. If the new schema is already defined, the consumer can release its own update prior to the producer release.
- Schema-encoded data is sent by a product deployed on customer hardware, such as a cell phone application that reports on user metrics. Updates can be made to the schema format for new producer releases, while maintaining the compatibility with previous releases.
- The consumer application may need to reprocess data in the event stream that was produced with an older schema version. Schema evolution ensures that the consumer can translate it to a familiar version. If backward compatibility is not followed, the consumer will only be able to read messages with the latest format.

Full compatibility

The union of forward compatibility and backward compatibility, this is the strongest guarantee and the one you should use whenever possible. You can always loosen the compatibility requirements at a later date, but it is often far more difficult to tighten them.

Code Generator Support

A *code generator* is used to turn an event schema into a class definition or equivalent structure for a given programming language. This class definition is used by the producer to create and populate new event objects. The producer is required by the compiler or serializer (depending on the implementation) to respect data types and populate all non-nullable fields that are specified in the original schema. The objects created by the producer are then converted into their serialized format and sent to the event broker, as shown in [Figure 3-1](#).

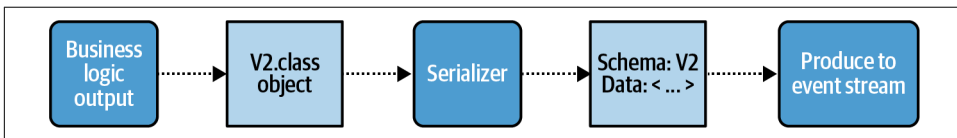


Figure 3-1. Producer event production workflow using a code generator

The consumer of the event data maintains its own version of the schema, which is often the same version as the producer's but could be an older or newer schema, depending on the usage of schema evolution. If full compatibility is being observed, the service can use any version of the schema to generate its definitions. The consumer reads the event and deserializes it using the schema version that it was encoded with. The event format is stored either alongside the message, which can be prohibitively expensive at scale, or in a schema registry and accessed on-demand (see [“Schema Registry” on page 241](#)). Once deserialized into its original format, the event can be converted to the version of the schema supported by the consumer. Evolution rules come into play at this point, with defaults being applied to missing fields, and unused fields dropped completely. Finally, the data is converted into an object based on the schema-generated class. At this point, the consumer's business logic may begin its operations. This process is shown in [Figure 3-2](#).

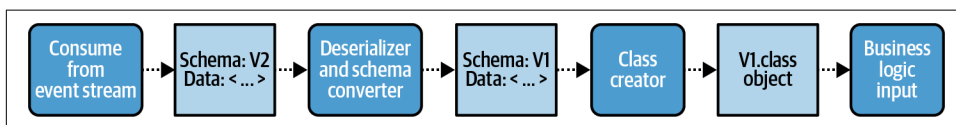


Figure 3-2. Consumer event consumption and conversion workflow using a code generator; note that the consumer converts events from schema v2, as created by the producer, to v1, the schema format used by the consumer

The biggest benefit of code generator support is being able to write your application against a class definition in the language of your choice. If you are using a compiled language, the code generator provides compiler checks to ensure that you aren't mishandling event types or missing the population of any given non-null data field. Your code will not compile unless it adheres to the schema, and therefore your application will not be shipped without adhering to the schema data definition. Both compiled and noncompiled languages benefit from having a class implementation to code against. A modern IDE will notify you when you're trying to pass the wrong types into a constructor or setter, whereas you would receive no notification if you're instead using a generic format such as an object key/value map. Reducing the risk of mishandling data provides for much more consistent data quality across the ecosystem.

Breaking Schema Changes

There are times when the schema definition must change in a way that results in a breaking evolutionary change. This can happen for a number of reasons, including evolving business requirements that alter the model of the original domain, improper scoping of the original domain, and human error while defining the schema. While the producing service can be fairly easily changed to accommodate the new schema, the impacts to downstream consumers vary and need to be taken into account.



The most important thing when dealing with breaking schema changes is to communicate early and clearly with downstream consumers. Ensure that any migration plans have the understanding and approval of everyone involved and that no one is caught unprepared.

While it may seem heavy-handed to require intense coordination between producers and consumers, the renegotiation of the data contract and the alteration of the domain model require buy-in from everyone. Aside from renegotiating the schema, you need to take some additional steps to accommodate the new schema and new event streams that are created from it. Breaking schema changes tend to be quite impactful for entities that exist indefinitely, but less so for events that expire after a given period of time.

Accommodating breaking schema changes for entities

Breaking changes to an entity schema are fairly rare, as this circumstance typically requires a redefinition of the original domain model such that the current model cannot simply be extended. New entities will be created under the new schema, while previous entities were generated under the old schema. This divergence of data definition leaves you with two choices:

- Contend with both the old and new schemas.
- Re-create all entities in the new schema format (via migration, or by re-creating them from source).

The first option is the easiest for the producer, but it simply pushes off the resolution of the different entity definitions onto the consumer. This contradicts the goal of *reducing* the need for consumers to interpret the data individually and increases the risk of misinterpretation, inconsistent processing between services, and significantly higher complexity in maintaining systems.



The reality is that the consumer will never be in a better position than the producer for resolving divergent schema definitions. It is bad practice to defer this responsibility to the consumer.

The second option is more difficult for the producer, but ensures that the business entities, both old and new, are redefined consistently. In practice, the producer must reprocess the source data that led to the generation of the old entities and apply the new business logic to re-create the entities under the new format. This approach forces the organization as a whole to resolve what these entities mean and how they should be understood and used by producer and consumer alike.



Leave the old entities under the old schema in their original event stream, because you may need them for reprocessing validation and forensic investigations. Produce the new and updated entities using the new schema to a new stream.

Accommodating breaking schema changes for events

Nonentity events tend to be simpler to deal with when you are incorporating breaking changes. The simplest option is to create a new event stream and begin producing the new events to that stream. The consumers of the old stream must be notified so that they can register themselves as consumers of the new event stream. Each consuming service must also account for the divergence in business logic between the two event definitions.



Don't mix different event types in an event stream, especially event types that are evolutionarily incompatible. Event stream overhead is cheap, and the logical separation is important in ensuring that consumers have full information and explicit definitions when dealing with the events they need to process.

Given that the old event stream no longer has new events being produced to it, the consumers of each consuming service will eventually catch up to the latest record. As time goes on, the stream's retention period will eventually result in a full purging of the stream, at which point all consumers can unregister themselves and the event stream can be deleted.

Selecting an Event Format

While there are many options available for formatting and serializing event data, data contracts are best fulfilled with strongly defined formats such as Avro, Thrift, or Protobuf. Some of the most popular event broker frameworks have support for serializing and deserializing events encoded with these formats. For example, both [Apache Kafka](#) and [Apache Pulsar](#) support JSON, Protobuf, and Avro schema formats. The mechanism of support for both of the technologies is the schema registry, which is covered in more detail in [“Schema Registry” on page 241](#). Though a detailed evaluation and comparison of these serialization options is beyond the scope of this book, there are a number of online resources available that can help you decide among these particular options.

You may be tempted to choose a more flexible option in the form of plain-text events using simple key/value pairs, which still offers some structure but provides no explicit schema or schema evolution frameworks. Proceed cautiously with this approach,

however, as it can compromise microservices' ability to remain isolated from one another via a strong data contract, requiring far more interteam communication.



Unstructured plain-text events usually become a burden to both the producer and the consumer, particularly as use cases and data changes over time. As mentioned, I recommend instead choosing a strongly defined, explicit schema format that supports controlled schema evolution, such as Apache Avro or Protobuf. I do not recommend JSON, as it does not provide full-compatibility schema evolution.

Designing Events

There are a number of best practices to follow when you are creating event definitions, as well as several anti-patterns to avoid. Keep in mind that as the number of architectures powered by event-driven microservices expands, so does the number of event definitions. Well-designed events will minimize the otherwise repetitive pain points for both consumers and producers. With that being said, none of the following are hard-and-fast rules. You can break them as you see fit, though I recommend that you think very carefully about the full scope of implications and the tradeoffs for your problem space before proceeding.

Tell the Truth, the Whole Truth, and Nothing but the Truth

A good event definition is not simply a message indicating that *something* happened, but rather the complete description of *everything* that happened during that event. In business terms, this is the resultant data that is produced when input data is ingested and the business logic is applied. This output event must be treated as the single source of truth and must be recorded as an immutable fact for consumption by downstream consumers. It is the full and total authority on what actually occurred, and consumers should not need to consult any other source of data to know that such an event took place.

Use a Singular Event Definition per Stream

An event stream should contain events representing a single logical event. It is not advisable to mix different types of events within an event stream, because doing so can muddle the definitions of what the event is and what the stream represents. It is difficult to validate the schemas being produced, as new schemas may be added dynamically in such a scenario. Though there are special circumstances where you may wish to ignore this principle, the vast majority of event streams produced and consumed within your architectural workflow should each have a strict, single definition.

Use the Narrowest Data Types

Use the narrowest types for your event data. This lets you rely on the code generators, language type checking (if supported), and serialization unit tests to check the boundaries of your data. It sounds simple, but there are many cases where ambiguity can creep in when you don't use the proper types. Here are a few easily avoidable real-world examples:

Using string to store a numeric value

This requires the consumer to parse and convert the string to a numeric value and often comes up with GPS coordinates. This is error prone and subject to failures, especially when a null value or an empty string is sent.

Using integer as a boolean

While 0 and 1 can be used to denote false and true, respectively, what does 2 mean? How about -1?

Using string as an enum

This is problematic for producers, as they must ensure that their published values match an accepted pseudo-enum list. Typos and incorrect values will inevitably be introduced. A consumer interested in this field will need to know the range of possible values, and this will require talking to the producing team, unless it's specified in the comments of the schema. In either case, this is an *implicit* definition, since the producers are not guarded against any changes to the range of values in the string. This whole approach is simply bad practice.

Enums are often avoided because producers fear creating a new enum token that isn't present in the consumer's schema. However, the consumer has a responsibility to consider enum tokens that it does not recognize, and determine if it should process them using a default value or simply throw a fatal exception and halt processing until someone can work out what needs to be done. Both Protobuf and Avro have elegant ways of handling unknown enum tokens and should be used if either is selected for your event format.

Keep Events Single-Purpose

One common anti-pattern is adding a type field to an event definition, where different type values indicate specific subfeatures of the event. This is generally done for data that is "similar yet different" and is often a result of the implementer incorrectly identifying the events as single-purpose. Though it may seem like a time-saving measure or a simplification of a data access pattern, overloading events with type parameters is rarely a good idea.

There are several problems with this approach. Each type parameter value usually has a fundamentally different business meaning, even if its technical representation is

nearly identical to the others. It is also possible for these meanings to change over time and for the scope that an event covers to creep. Some of these types may require the addition of new parameters to track type-specific information, whereas other types require separate parameters. Eventually you could have a situation where there are several very distinct events all inhabiting the same event schema, making it difficult to reason about what the event truly represents.

This complexity affects not only the developers who must maintain and populate these events, but also the data's consumers, who need to have a consistent understanding about what data is published and why. If the data contract changes, they expect to be able to isolate themselves from those changes. Adding extra field types requires them to filter for only data that they care about. There is a risk that the consumer will fail to fully understand the various meanings of the types, leading to incorrect consumption and logically wrong code. For each consumer, additional processing must also be done to discard events that aren't relevant to that consumer.



It is very important to note that adding type fields does not reduce or eliminate the underlying complexity inherent in the data being produced. In fact, this complexity is merely shifted from multiple distinct event streams with distinct schemas to a union of all the schemas merged into one event stream. It could be argued that this actually *increases* the complexity. Future evolution of the schema becomes more difficult, as does maintaining the code that produces the events.

Remember the principles of the data contract definition. Events should be related to a single business action, not a generic event that records large assortments of data. If it seems like you need a generic event with various type parameters, that's usually a tell-tale sign that your problem space and bounded context is not well defined.

Example: Overloading event definitions

Imagine a simple website where a user can read a book or watch a movie. When the user first engages the website, say by opening the book or starting the movie, a back-end service publishes an event of this engagement, named `ProductEngagement`, into an event stream. The data structure of this cautionary tale event may look something like this:

```
TypeEnum: Book, Movie
ActionEnum: Click

ProductEngagement {
    productId: Long,
    productType: TypeEnum,
    actionType: ActionEnum
}
```

Now imagine a new business requirement comes in: you need to track who watched the movie trailer before watching the movie. There are no previews for books, and though a boolean would suit the movie-watching case, it must be nullable to allow for book engagements.

```
ProductEngagement {
  productId: Long,
  productType: TypeEnum,
  actionType: ActionEnum,
  //Only applies to type=Movie
  watchedPreview: {null, Boolean}
}
```

At this point, `watchedPreview` has nothing to do with Books, but it's added into the event definition anyway since we're already capturing product engagements this way. If you're feeling particularly helpful to your downstream consumers, you can add a comment in the schema to tell them that this field is only related to `type=Movie`.

Another new business requirement comes in: you need to track users who place a bookmark in their book, and log what page it is on. Again, because there is only a single defined structure of events for product engagements, your course of action is constrained to adding a new action entity (Bookmark) and adding a nullable `PageId` field.

```
TypeEnum: Book, Movie
ActionEnum: Click, Bookmark

ProductEngagement {
  productId: Long,
  productType: TypeEnum,
  actionType: ActionEnum,
  //Only applies to productType=Movie
  watchedPreview: {null, Boolean},
  //Only applies to productType=Book,actionType=Bookmark
  pageId: {null, Int}
}
```

As you can see by now, just a few changes in business requirements can greatly complicate a schema that is trying to serve multiple business purposes. This adds complexity for the producer and the consumer of the data, as they both must check for the validity of the data logic. The data to be collected and represented will always be complex, but by following single responsibility principles you can decompose the schema into something more manageable. Let's see what this example would look like if you split up each schema according to single responsibilities:

```
MovieClick {
  movieId: Long,
  watchedPreview: Boolean
}
```

```
BookClick {  
    bookId: Long  
}  
  
BookBookmark {  
    bookId: Long,  
    pageId: Int  
}
```

The `productType` and `actionType` enumerations are now gone, and the schemas have been flattened out accordingly. There are now three event definitions instead of just a single one, and while the schema definition count has increased, the internal complexity of each schema is greatly reduced. Following the recommendation of one event definition per stream would see the creation of a new stream for each event type. Event definitions would not drift over time, the triggering logic would not change, and consumers could be secure in the stability of the single-purpose event definition.

The takeaway from this example is not that the original creator of the event definition made a mistake. In fact, at the time, the business cared about *any* product engagements but no *specific* product engagement, so the original definition is quite reasonable. As soon as the business requirements changed to include tracking movie-specific events, the owner of the service needed to re-evaluate whether the event definition was still serving a single purpose, or if it was instead now covering multiple purposes. Due to the business requiring lower-level details of an event, it became clear that, while the event could serve multiple purposes, it soon would become complex and unwieldy to do so.



Avoid adding type fields in your events that overload the meaning of the event. This can cause significant difficulty in evolving and maintaining the event format.

Take some time to consider how your schemas may evolve. Identify the main business purpose of the data being produced, the scope, the domain, and whether you're building it as single-purpose. Validate that the schemas accurately reflect business concerns, especially for systems that cover a broad scope of business function responsibility. It could be that the business scope and the technical implementation are misaligned. Finally, evolving business requirements may require you to revisit the event definitions and potentially change them beyond just incremental definitions of a single schema. Events may need to be split up and redefined completely should sufficient business changes occur.

Minimize the Size of Events

Events work well when they're small, well defined, and easily processed. Large events can and do happen, though. Generally these larger events represent a lot of contextual information. Perhaps they comprise many data points that are related to the given event, and are simply a very large measurement of something that occurred.

There are several considerations when you're looking at a design that produces a very large event. Make sure that the data is directly related and relevant to the event. Additional data may have been added to an event "just in case," but it may not be of any real use to the downstream consumers. If you find that all the event data is indeed directly related, take a step back and look at your problem space. Does your microservice require access to the data? You may want to evaluate the bounded context to see if the service is performing a reasonable amount of work. Perhaps the service could be reduced in scope with additional functionality split off into its own service.

This scenario is not always avoidable, though—some event processors produce very large output files (perhaps a large image) that are much too big to fit into a single message of an event stream. In these scenarios you can use a pointer to the actual data, but do this sparingly. This approach adds risk in the form of multiple sources of truth and payload mutability, as an immutable ledger cannot ensure the preservation of data outside of its system.

Involve Prospective Consumers in the Event Design

When designing a new event, it is important to involve any anticipated consumers of this data. Consumers will understand their own needs and anticipated business functions better than the producers and may help in clarifying requirements. Consumers will also get a better understanding of the data coming their way. A joint meeting or discussion can shake out any issues around the data contract between the two systems.

Avoid Events as Semaphores or Signals

Avoid using events as a semaphore or a signal. These events simply indicate that something has occurred without being the single source of truth for the results.

Consider a very simple example where a system outputs an event indicating that work has been completed for an arbitrary job. Although the event itself indicates the work is done, the actual result of the work is not included in the event. This means that to consume this event properly, you must find where the completed work actually resides. Once there are two sources of truth for a piece of data, consistency problems arise.

Summary

Asynchronous event-driven architectures rely heavily upon event quality. High-quality events are explicitly defined with an evolvable schema, have well-defined triggering logic, and include full schema definitions with comments and documentation. Implicit schemas, while easier to implement and maintain for the producer, offload much of the interpretation work onto the consumer. They are also more prone to unexpected failures due to missing event data and unintentional changes. Explicit schemas are an essential component for widespread adoption of event-driven architectures, particularly as an organization grows and it becomes impossible to communicate tribal knowledge organization-wide.

Event definitions should be narrow and closely focused on the domain of the event. An event should represent a specific *business* occurrence and contain the appropriate data fields to record what has happened. These events form the official narrative about business operations and can be consumed by other microservices for their own needs.

Schema evolution is a very important aspect of explicit schemas, as it allows for a controlled mechanism of change for the event domain model. It is common for a domain model to evolve, particularly as new business requirements emerge and the organization expands. Schema evolution allows producers and consumers to isolate themselves from changes that aren't essential to their operations, while permitting those services that *do* care about the changes to update themselves accordingly.

In some cases schema evolution is not possible, and a breaking change must occur. The producer and consumer stakeholders must communicate the reasons behind the breaking changes and come together to redefine the domain model going forward. Migration of old events may or may not be necessary.

Integrating Event-Driven Architectures with Existing Systems

Transitioning an organization to an event-driven architecture requires the integration of existing systems into the ecosystem. Your organization may have one or more monolithic relational database applications. Point-to-point connections between various implementations are likely to exist. Perhaps there are already event-like mechanisms for transferring bulk data between systems, such as regular syncing of database dumps via an intermediary file store location. In the case that you are building an event-driven microservice architecture from the ground up and have no legacy systems, great! You can skip this section (though perhaps you should consider that EDM may not be right for your new project). However, if you have existing legacy systems that need to be supported, read on.

In any business domain, there are entities and events that are commonly required across multiple subdomains. For example, an ecommerce retailer will need to supply product information, prices, stock, and images to various bounded contexts. Perhaps payments are collected by one system but need to be validated in another, with analytics on purchase patterns performed in a third system. Making this data available in a central location as the new single source of truth allows each system to consume it as it becomes available. Migrating to event-driven microservices requires making the necessary business domain data available in the event broker, consumable as event streams. Doing so is a process known as *data liberation*, and involves *sourcing* the data from the existing systems and state stores that contain it.

Data produced to an event stream can be accessed by any system, event-driven or otherwise. While event-driven applications can use streaming frameworks and native consumers to read the events, legacy applications may not be able to access them as easily due to a variety of factors, such as technology and performance limitations. In

this case, you may need to *sink* the events from an event stream into an existing state store.

There are a number of patterns and frameworks for sourcing and sinking event data. For each technique, this chapter will cover why it's necessary, how to do it, and the tradeoffs associated with different approaches. Then, we'll review how data liberation and sinking fit in to the organization as a whole, the impacts they have, and ways to structure your efforts for success.

What Is Data Liberation?

Data liberation is the identification and publication of cross-domain data sets to their corresponding event streams and is part of a *migration strategy* for event-driven architectures. Cross-domain data sets include any data stored in one data store that is required by other external systems. Point-to-point dependencies between existing services, and data stores often highlight the cross-domain data that should be liberated, as shown in [Figure 4-1](#), where three dependent services are querying the legacy system directly.

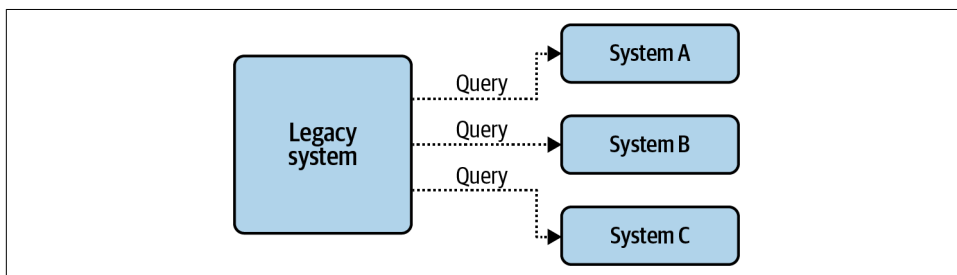


Figure 4-1. Point-to-point dependencies, accessing data directly from the underlying service

Data liberation enforces two primary features of event-driven architecture: the single source of truth and the elimination of direct coupling between systems. The liberated event streams allow new event-driven microservices to be built as consumers, with existing systems migrated in due time. Reactive event-driven frameworks and services may now be used to consume and process data, and downstream consumers are no longer required to directly couple on the source data system.

By serving as a single source of truth, these streams also standardize the way in which systems across the organization access data. Systems no longer need to couple directly to the underlying data stores and applications, but instead can couple solely on the data contracts of event streams. The post-liberation workflow is shown in [Figure 4-2](#).

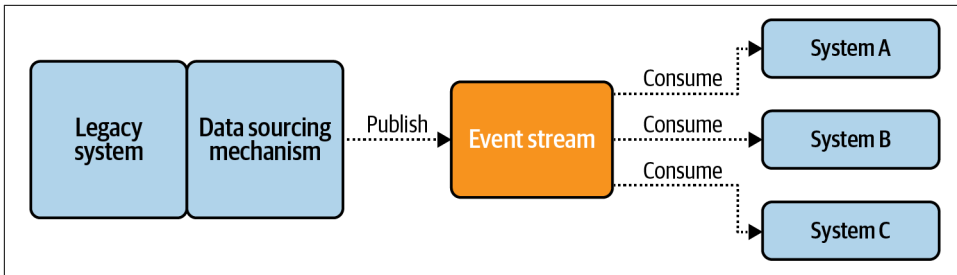


Figure 4-2. Post-data-liberation workflow

Compromises for Data Liberation

A data set and its liberated event stream must be kept fully in sync, although this requirement is limited to eventual consistency due to the latency of event propagation. A stream of liberated events must materialize back into an exact replica of the source table, and this property is used extensively for event-driven microservices (as covered in [Chapter 7](#)). In contrast, legacy systems do not rebuild their data sets from any event streams, but instead typically have their own backup and restore mechanisms and read absolutely nothing back from the liberated event stream.

In the perfect world, all state would be created, managed, maintained, and restored from the single source of truth of the event streams. Any shared state should be published to the event broker *first* and materialized back to any services that need to materialize the state, including the service that produced the data in the first place, as shown in [Figure 4-3](#).

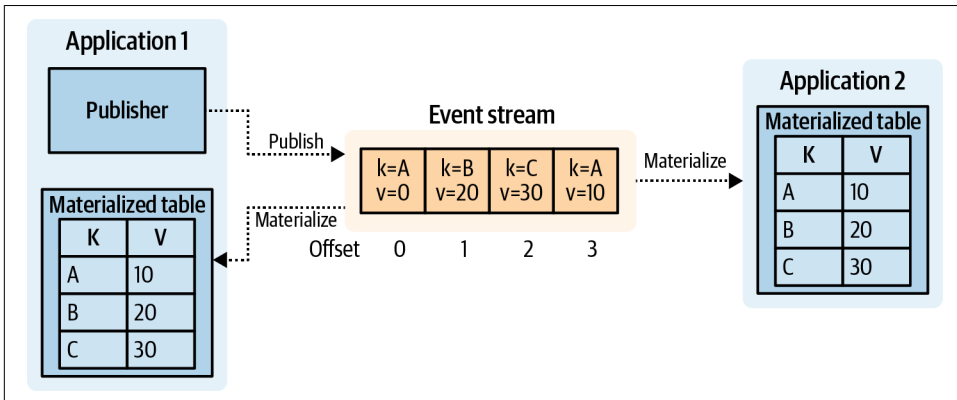


Figure 4-3. Publish to stream before materializing

While the ideal of maintaining state in the event broker is accessible for new microservices and refactored legacy applications, it is *not* necessarily available or practical for all applications. This is particularly true for services that are unlikely to ever be

refactored or changed beyond initial integration with change-data capture mechanisms. Legacy systems can be both extremely important to the organization and prohibitively difficult to refactor, with the worst offenders being considered **a big ball of mud**. Despite the complexity of a system, their internal data will still need to be accessed by other new systems. While refactoring may absolutely be desirable, there are a number of issues that prevent this from happening in reality:

Limited developer support

Many legacy systems have minimal developer support and require low-effort solutions to generate liberated data.

Expense of refactoring

Reworking the preexisting application workflows into a mix of asynchronous event-driven and synchronous MVC (Model-View-Controller) web application logic may be prohibitively expensive, especially for complex legacy monoliths.

Legacy support risk

Changes made to legacy systems may have unintended consequences, especially when the system's responsibilities are unclear due to technical debt and unidentified point-to-point connections with other systems.

There is an opportunity for compromise here. You can use data liberation patterns to extract the data out of the data store and create the necessary event streams. This is a form of unidirectional event-driven architecture, as the legacy system will *not* be reading back from the liberated event stream, as shown in **Figure 4-3**. Instead, the fundamental goal is to keep the internal data set synchronized with the external event stream through strictly controlled publishing of event data. The event stream will be eventually consistent with the internal data set of the legacy application, as shown in **Figure 4-4**.

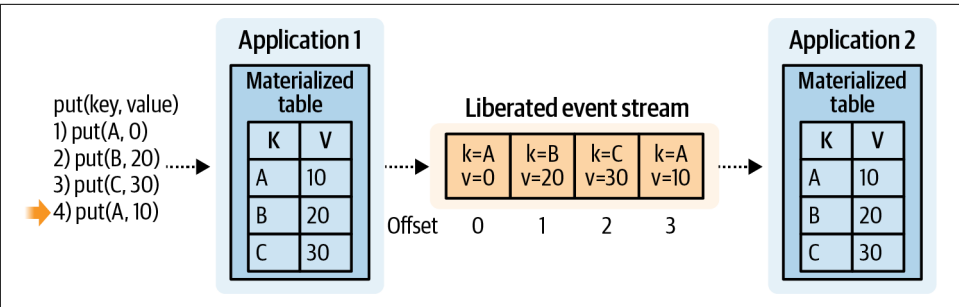


Figure 4-4. Liberating and materializing state between two services

Converting Liberated Data to Events

Liberated data, much like any other event, is subject to the same recommendations of schematization that were introduced in [Chapter 3](#). One of the properties of a well-defined event stream is that there is an explicitly defined and evolutionarily compatible schema for the events it contains. You should ensure that consumers have basic data quality guarantees as part of the data contract defined by the schema. Changes to the schema can only be made according to evolutionary rules.



Use the same standard format for both liberated event data and native event data across your organization.

By definition, the data that is most relevant and used across the business is the data that is most necessary to liberate. Changes made to the data definitions of the source, such as creating new fields, altering existing ones, or dropping others, can result in dynamically changing data being propagated downstream to consumers. Failing to use an explicitly defined schema for liberated data will force downstream consumers to resolve any incompatibilities. This is extremely problematic for the provision of the single source of truth, as downstream consumers should not be attempting to parse or interpret data on their own. It is extremely important to provide a reliable and up-to-date schema of the produced data and to carefully consider the evolution of the data over time.

Data Liberation Patterns

There are three main data liberation patterns that you can use to extract data from the underlying data store. Since liberated data is meant to form the new single source of truth, it follows that it must contain the entire set of data from the data store. Additionally, this data must be kept up to date with new insertions, updates, and deletes.

Query-based

You extract data by querying the underlying state store. This can be performed on any data store.

Log-based

You extract data by following the append-only log for changes to the underlying data structures. This option is available only for select data stores that maintain a log of the modifications made to the data.

Table-based

In this pattern, you first push data to a table used as an output queue. Another thread or separate process queries the table, emits the data to the relevant event

stream, and then deletes the associated entries. This method requires that the data store support both transactions and an output queue mechanism, usually a standalone table configured for use as a queue.

While each pattern is unique, there is one commonality among the three. Each should produce its events in sorted timestamp order, using the source record's most recent `updated_at` time in its output event record header. This will generate an event stream timestamped according to the event's occurrence, not the time that the producer published the event. This is particularly important for data liberation, as it accurately represents when events actually happened in the workflow. Timestamp-based interleaving of events is discussed further in [Chapter 6](#).

Data Liberation Frameworks

One method of liberating data involves the usage of a dedicated, centralized framework to extract data into event streams. Examples of centralized frameworks for capturing event streams include [Kafka Connect \(exclusively for the Kafka platform\)](#), [Apache Gobblin](#), and [Apache NiFi](#). Each framework allows you to execute a query against the underlying data set with the results piped through to your output event streams. Each option is also scalable, such that you can add further instances to increase the capacity for executing change-data capture (CDC) jobs. They support various levels of integration with the [schema registry offered by Confluent \(Apache Kafka\)](#), but customization can certainly be performed to support other schema registries. See [“Schema Registry” on page 241](#) for more information.

Not all data liberation processes require a dedicated framework, and many systems are better suited to taking direct ownership of their own event stream data production. In fact, these frameworks inadvertently encourage data access anti-patterns. One of the most common anti-patterns is the exposure of internal data models to external systems, further increasing coupling instead of decreasing it, as is one of the major benefits of event-driven architectures. This will be covered further in the remainder of the chapter.

Liberating Data by Query

Query-based data liberation involves querying the data store and emitting selected results to an associated event stream. A client is used to request the specific data set from the data store using the appropriate API, SQL, or SQL-like language. A data set must be bulk-queried to provide the history of events. Periodic updates then follow, ensuring that changes are produced to the output event stream.

There are several types of queries used in this pattern.

Bulk Loading

Bulk loading queries and loads all of the data from the data set. Bulk loads are performed when the entire table needs to be loaded at each polling interval, as well as prior to ongoing incremental updates.

Bulk loading can be expensive, as it requires obtaining the entire data set from the data store. For smaller data sets this tends not to be a problem, but large data sets, especially those with millions or billions of records, may be difficult to obtain. For querying and processing very large data sets I recommend you research best practices for your particular data store, since these can vary significantly with implementation.

Incremental Timestamp Loading

With incremental timestamp loading, you query and load all data since the highest timestamp of the previous query's results. This approach uses an `updated-at` column or field in the data set that keeps track of the time when the record was last modified. During each incremental update, only records with `updated-at` timestamps later than the last processed time are queried.

Autoincrementing ID Loading

Autoincrementing ID loading involves querying and loading all data larger than the last value of the ID. This requires a strictly ordered autoincrementing `Integer` or `Long` field. During each incremental update, only records with an ID larger than the last processed ID are queried. This approach is often used for querying tables with immutable records, such as when using the outbox tables (see [“Liberating Data Using Change-Data Capture Logs” on page 61](#)).

Custom Querying

A custom query is limited only by the client querying language. This approach is often used when the client requires only a certain subset of data from a larger data set, or when joining and denormalizing data from multiple tables to avoid over-exposure of the internal data model. For instance, a user could filter business partner data according to a specific field, where each partner's data is sent to its own event stream.

Incremental Updating

The first step of any incremental update is to ensure that the necessary timestamp or autoincrementing ID is available in the records of your data set. There must be a field that the query can use to filter out records it has already processed from those it has yet to process. Data sets that lack these fields will need to have them added, and the data store will need to be configured to populate the necessary `updated_at`

timestamp or the autoincrementing ID field. If the fields cannot be added to the data set, then incremental updates will not be possible with a query-based pattern.

The second step is to determine the frequency of polling and the latency of the updates. Higher-frequency updates provide lower latency for data updates downstream, though this comes at the expense of a larger total load on the data store. It's also important to consider whether the interval between requests is sufficient to finish loading all of the data. Beginning a new query while the old one is still loading can lead to race conditions, where older data overwrites newer data in the output event streams.

Once the incremental update field has been selected and the frequency of updates determined, the final step is to perform a single bulk load before enabling incremental updates. This bulk load must query and produce all of the existing data in the data set prior to further incremental updates.

Benefits of Query-Based Updating

Query-based updating has a number of advantages, including:

Customizability

Any data store can be queried, and the entire range of client options for querying is available.

Independent polling periods

Specific queries can be executed more frequently to meet tighter SLAs (service-level agreements), while other more expensive queries can be executed less frequently to save resources.

Isolation of internal data models

Relational databases can provide isolation from the internal data model by using views or materialized views of the underlying data. This technique can be used to hide domain model information that should not be exposed outside of the data store.



Remember that the liberated data will be the single source of truth. Consider whether any concealed or omitted data should instead be liberated, or if the source data model needs to be refactored. This often occurs during data liberation from legacy systems, where business data and entity data have become intertwined over time.

Drawbacks of Query-Based Updating

There are some downsides to query-based updating as well:

Required updated-at timestamp

The underlying table or namespace of events to query must have a column containing their `updated-at` timestamp. This is essential for tracking the last update time of the data and for making incremental updates.

Untraceable hard deletions

Hard deletions will not show up in the query results, so tracking deletions is limited to flag-based soft deletions, such as a boolean `is_deleted` column.

Brittle dependency between data set schema and output event schema

Data set schema changes may occur that are incompatible with downstream event format schema rules. Breakages are increasingly likely if the liberation mechanism is separate from the code base of the data store application, which is usually the case for query-based systems.

Intermittent capture

Data is synced only at polling intervals, and so a series of individual changes to the same record may only show up as a single event.

Production resource consumption

Queries use the underlying system resources to execute, which can cause unacceptable delays on a production system. This issue can be mitigated by the use of a read-only replica, but additional financial costs and system complexity will apply.

Variable query performance due to data changes

The quantity of data queried and returned varies depending on changes made to the underlying data. In the worst-case scenario, the entire body of data is changed each time. This can result in race conditions when a query is not finished before the next one starts.

Liberating Data Using Change-Data Capture Logs

Another pattern for liberating data is using the data store's underlying *change-data* capture logs (*binary* logs in MySQL, *write-ahead* logs for PostgreSQL) as the source of information. This is an append-only data logging structure that details everything that has happened to the tracked data sets over time. These changes include the creation, deletion, and updating of individual records, as well as the creation, deletion, and altering of the individual data sets and their schemas.

The technology options for change-data capture are narrower than those for query-based capturing. Not all data stores implement an immutable logging of changes, and